



LARGE LANGUAGE MODELS

AND AI TOOLING



IN THIS PRESENTATION

- Large Language Model
- LLM best Practices
- Gen AI for Developers
- AI Pair Programming
- Copilot
- GenAI for Code Generation
- GenAI for Documentation
- Optimization & Analysis

LARGE LANGUAGE MODEL(LLM)

- Large Language Models are advanced machine learning models trained on massive collections of text data
- These models are trained to understand, generate, and reason with human language
- They use probabilistic patterns in language to produce responses, answer questions, write code, and summarise information
- LLMs form the foundation of modern GenAI systems
 - Examples include GPT, BERT, Claude, Llama, Copilot, and Codeium

LLM BEST PRACTICES

- Use clear instructions
- Provide appropriate constraints
- Use thoughtful validation of outputs
- Provide sufficient context
- Limit response scope
- Avoid ambiguous prompts
- Always verify results before using them

GENAI FOR DEVELOPERS

- GenAI provides developers with tools that can accelerate coding tasks
 - Writing boilerplate code
 - Explaining unfamiliar codebases
 - Debugging errors
 - Suggesting improvements
- Acts as a productivity multiplier, NOT a replacement for developer expertise

AI PAIR PROGRAMMING

- AI pair programming involves working alongside an AI assistant that can perform some tasks to help with developer work
 - Suggest code
 - Explain logic
 - Troubleshoot problems in real time
- Similar to human pair programming
- Can help reduce cognitive load and improve development efficiency

COPILOT

- GitHub Copilot is an AI-powered assistant that integrates directly into development environments to provide real-time code suggestions
- It uses context from the current file and project to generate relevant code, comments, and test cases
- Copilot helps developers code faster and more efficiently

GENAI FOR CODE GENERATION

- GenAI can automatically produce code snippets, functions, and even entire modules based on natural language descriptions
- This can significantly speed up development
- BUT generated code must always be reviewed for correctness, security, and maintainability
- GenAI is not always consistent with naming conventions between generated snippets
- It cannot take into account the entire scope of large projects

GENAI FOR DOCUMENTATION

- GenAI can assist with creating and maintaining technical documentation
- It does so by generating:
 - Comments
 - README files
 - API descriptions
 - Usage examples
- This helps improve code readability and knowledge sharing while reducing the time devs spend on manual documentation tasks

OPTIMIZATION AND ANALYSIS

- GenAI can support optimization and analysis by identifying inefficiencies, suggesting refactors, analyzing performance bottlenecks, and explaining complex logic
- This helps developers to improve code quality and system performance
- Devs get a deeper understanding of existing implementations

AI TOOLING

- Developers use **Copilot** to generate boilerplate code and unit tests
 - Developer Assistants - GitHub Copilot, Codeium
- Data teams use **LLMs** to translate business questions into SQL queries
 - Large Language Models - GPT, Claude, LLaMA
- Analysts use **GenAI** to summarize dashboards and trends
- Engineers use **AI assistants** to explain legacy codebases
 - Data & Analytics - BigQuery ML, Azure OpenAI, AWS Bedrock
 - MLOps & AI Ops - MLflow, Vertex AI, SageMaker